ACADEMIC CAPSTONE PROJECT REPORT

TimeTrade: Student Time-Exchange Board

A Full-Stack Peer-to-Peer Academic Assistance & Time Slot Reservation Platform
Engineered with MERN Stack (MongoDB, Express.js, React 18, Node.js) and Tailwind CSS

Course / Degree: Bachelor of Computer Applications (BCA) - Semester V / Final Capstone
Project Module: BCA • Frontend Application Development Using REACT & REST API Backend
Author / Student: Abuzer [Student Candidate]
Roll / Exam No.: BCA-5TH-SEM-PROJECT-11
Academic Mentor: Department Faculty Guide & Evaluation Committee
Department: Department of Computer Science & Information Technology

Academic Year: 2025 – 2026
Submission Date: October 2026

Verified for Academic & Industry-Level Full-Stack Engineering Compliance
Production GitHub Repository Standard • Dual-Module Architecture • MIT License

1. Executive Summary & Abstract

The TimeTrade platform is an enterprise-grade, peer-to-peer student collaboration and time-banking web application designed to eliminate academic assistance friction within university campuses. Higher education students frequently encounter localized, short-duration academic roadblocks—ranging from elusive code debugging in laboratory assignments, handwritten lecture note review for semester midterms, and capstone presentation defense coaching, to complex algorithmic problem solving. While traditional commercial tutoring is expensive, slow, and transactional, peer student networks possess immense, untapped, complementary expertise. However, peer learning typically falters due to the lack of an organized exchange mechanism, mutual commitment, and accountability.

TimeTrade solves this paradigm by formalizing reciprocal 'Time Banking': students offer short, high-yield help sessions in topics they have mastered and in return request equivalent help slots from peers in topics where they need support, entirely free of monetary exchange. The technical foundation is built on an enterprise-grade dual-module architecture comprising a decoupled React 18 Single Page Application (SPA) bundled via Vite and Tailwind CSS, and a secure Node.js/Express.js RESTful API backed by MongoDB Mongoose schemas.

> **NOTE: Core Value Proposition**
> *TimeTrade establishes an equitable campus economy: 1 hour spent reviewing web technologies theory notes earns 1 hour of one-on-one debugging or mock capstone defense preparation, driving cooperative peer learning.*

Key Technical & Functional Achievements

- **Dual-Module Decoupled Architecture:** Clean isolation between client (/client) and server (/server) with separate dependency graphs, independent linting, and automated environment configurations.
- **Two-Sided Request State Machine:** Deterministic slot reservation workflow supporting Open, Booked, Completed, and Cancelled session lifecycles, coupled with automatic conflict resolution upon host acceptance.
- **Multi-Tiered Security Hardening:** Implementation of Helmet.js HTTP headers, strict CORS origin whitelisting, bcryptjs salted password hashing, express-validator input sanitization, and express-rate-limit brute force mitigation.
- **Unified 4-Tab Student Command Center:** Tabbed management dashboard tracking Active Offerings, Incoming Inquiries, Outgoing Booking Requests, and an immutable Completed History Log.

2. Problem Statement & Project Concept

2.1 The Academic Collaboration Dilemma

In modern computer science and BCA curricula, students are confronted with an intensive mix of theoretical coursework (Theory of Computation, Database Management, Operating Systems) and rapid-paced practical applications (MERN Stack, Responsive Web Design, REST API engineering). The resulting challenges include:

- **Fragmented Communication Channels:** Student interactions occur across unindexed instant messaging apps (WhatsApp, Telegram) where academic requests are easily buried, lost, or ignored.

- **One-Sided Assistance Fatigue:** High-performing students often experience burnout when peers repeatedly ask for assistance without reciprocity, leading to hesitation in offering help.
- **Exam Revision & Notes Bottlenecks:** Students who miss lectures lack vetted, peer-curated revision sheets and past-year question summaries before examinations.
- **Pre-Defense Presentation Anxiety:** Capstone students frequently defend technical projects without realistic mock rehearsal, resulting in suboptimal pacing, communication breakdowns, and nervousness.

2.2 The TimeTrade Solution: Structured Reciprocal Exchange

TimeTrade introduces a structured, two-sided marketplace where student time is treated as a shared, valuable currency. Every student acts simultaneously as a 'Host' (offering skills or revision notes) and a 'Requester' (seeking targeted assistance). By substituting monetary payment with equal blocks of reciprocal time, TimeTrade democratizes campus learning and establishes a supportive academic community.

Category	Typical Scope	Target Student Need	Standard Duration
Coding	React, Node.js, Python, Java, Data Structures debugging	Assignment bug triage & code walkthroughs	30 – 60 Mins
Notes	Handwritten lecture summaries, exam cheat sheets, PYQs	Midterm & semester exam preparation	30 – 45 Mins
Presentation Practice	Capstone project defense rehearsal, slide feedback	Final year presentation polish & Q&A readiness	45 – 60 Mins
Academic Support	Discrete Math, Statistics, Automata proofs	Complex homework and proof derivations	45 – 90 Mins
Other	Git workflows, Linux CLI, Docker container setup	DevOps and technical toolchain pairing	30 – 45 Mins

3. System Requirements & Technical Architecture

3.1 Functional Requirements (FR)

- **FR-01 (Authentication):** Students must register with unique university email addresses, hash passwords via bcrypt, and maintain authenticated sessions using JSON Web Tokens (JWT).
- **FR-02 (Session Marketplace):** Public and authenticated users can view, search in real-time via keywords, and filter active time-exchange offerings across 5 academic categories.
- **FR-03 (Session Offer CRUD):** Logged-in students can create, view, modify, and delete their own session offerings with title, category, description, duration, and scheduled time.
- **FR-04 (Slot Request Submission):** Students can submit a booking request on any open session with an introductory note outlining specific learning goals.
- **FR-05 (Two-Sided Request Approval):** Session hosts can review incoming inquiries and Accept or Reject them. Accepting a request immediately updates session status to 'Booked' and automatically rejects conflicting requests.
- **FR-06 (Request Cancellation):** Either party can cancel an active booking; if an accepted booking is cancelled, the session availability status automatically returns to 'Open'.

- **FR-07 (Student Dashboard):** Unified multi-tab interface presenting 'My Offerings', 'Incoming Requests', 'My Sent Requests', and 'Completed History'.

3.2 Non-Functional Requirements (NFR)

- **NFR-01 (Security):** Zero plaintext password storage, protection against XSS and clickjacking via Helmet.js, brute-force mitigation via IP rate limiting, and CORS origin restriction.
- **NFR-02 (Performance):** Sub-100ms API response latency for marketplace queries facilitated by MongoDB compound indexes and lean payload serialization.
- **NFR-03 (Reliability):** Graceful connection lifecycle handlers and operational exception isolation ensuring server uptime does not crash on malformed inputs.
- **NFR-04 (Responsiveness & Accessibility):** 100% responsive user interface adapting to smartphones, tablets, and desktop displays with high-contrast text conforming to WCAG 2.1 AA.

3.3 Technology Stack Rationale

Layer	Selected Technology	Version	Engineering Justification
Frontend UI	React.js	18.3.1	Component-driven declarative UI, virtual DOM diffing, and seamless hook-based state management.
Build Tool	Vite	5.4.x	Instant Hot Module Replacement (HMR) powered by native ES modules, eliminating Webpack bundling latency.
Styling Engine	Tailwind CSS	3.4.x	Utility-first CSS delivering responsive styling without bloated stylesheet files; high-contrast student theme.
Routing & Guards	React Router DOM	6.28.x	Declarative client-side routing with nested layouts, ProtectedRoute auth guards, and PublicRoute guest guards.
HTTP Client	Axios	1.7.x	Promise-based HTTP client with request/response interceptors for dynamic Bearer token injection and global 401 handling.
Backend Runtime	Node.js	24.x LTS	Event-driven, asynchronous non-blocking I/O runtime optimal for concurrent API requests.
Web Framework	Express.js	4.21.x	Minimalist, robust REST API framework supporting modular routing, middleware pipelines, and error handling.
Database	MongoDB (Mongoose)	8.9.x	Document-oriented NoSQL storage facilitating flexible polymorphic session schemas and rich population queries.

3.4 High-Level System Architecture

The TimeTrade system adopts a layered client-server architecture. The React Single Page Application interacts with the Node.js/Express API gateway strictly through stateless RESTful JSON contracts. The server implements a layered service-oriented design:

- **Presentation & Network Interceptor:** React components make requests via an Axios singleton instance. The request interceptor dynamically injects 'Authorization: Bearer <token>' from localStorage, while the response interceptor monitors 401 Unauthorized codes to trigger session expiration.

- **Security & Gateway Pipeline:** Every request passes sequentially through Helmet HTTP security headers, CORS origin verification, express-rate-limit throttling, body parsing, and route parameter validation via express-validator.
- **Controller & Business Service Layer:** Controllers parse validated input and delegate domain logic to specialized services (authService, sessionService, bookingService), ensuring zero database operations are performed in route handlers.
- **Persistence & Mongoose Data Models:** Services interact with Mongoose models (User, Session, Booking), utilizing compound indexes, pre-save cryptographic hooks, and lean query projections.

4. Database Design & Schema Architecture

The database design emphasizes relational integrity within MongoDB's document model using Mongoose ObjectIds, population references, and compound indexes to prevent duplicate booking conflicts.

4.1 User Entity Schema (`User.js`)

The User collection stores authenticated student records. Passwords are automatically salted and hashed prior to persistence using a Mongoose pre-save hook. The password field is flagged with 'select: false' to prevent unintentional transmission in API query results.

Field Name	BSON Type	Constraints	Indexing	Functional Description
_id	ObjectId	Primary Key (Auto)	Unique Index	Unique student identifier generated by MongoDB.
name	String	Required, Trim, Max: 60	None	Student's full name displayed across offerings and requests.
email	String	Required, Unique, Lowercase	Unique Index	Institutional university email for login and notifications.
password	String	Required, Min: 6, Hashed	Hidden (select:false)	Bcrypt hashed password with 10 salt rounds.
department	String	Default: 'General Studies'	None	Academic major (e.g. Computer Science, Data Science).
avatarUrl	String	Optional (URL String)	None	URL to avatar image; falls back to Dicebear SVG avatar.
bio	String	Optional, Max: 240	None	Short description of student's academic interests and strengths.
createdAt / updatedAt	Date	Auto Timestamps	None	Audit timestamps tracking account creation and edits.

4.2 Session Offering Schema (`Session.js`)

The Session collection represents academic assistance offerings published by hosts. Compound indexes support rapid filtering by status and category, while a MongoDB text index enables full-text keyword searches across title, description, and skill tags.

Field Name	BSON Type	Constraints	Indexing	Functional Description

_id	ObjectId	Primary Key (Auto)	Unique Index	Unique session identifier.
creator	ObjectId	Required, Ref: 'User'	Indexed	Foreign reference to the User who created the offering.
title	String	Required, Trim, Max: 120	Text Index	Descriptive headline of the offered time session.
category	String	Enum: [Notes, Coding, Pres., Acad., Other]	Compound Index	Academic classification of the offering.
description	String	Required, Max: 1000	Text Index	Detailed syllabus, topic coverage, and prerequisites.
skillOffered	String	Optional, Trim, Max: 200	Text Index	Specific concept tags (e.g. 'React, Hooks, Vite').
durationMinutes	Number	Min: 10, Max: 240, Def: 45	None	Length of the session in minutes.
timeSlot	String	Required, Trim	None	Scheduled time or availability window (e.g. 'Thursday 4 PM').
status	String	Enum: [Open, Booked, Completed, Cancelled]	Compound Index	Current availability state (Default: 'Open').
createdAt / updatedAt	Date	Auto Timestamps	Compound Index	Chronological audit timestamps.

4.3 Booking & Request Schema (`Booking.js`)

The Booking collection governs the two-sided slot reservation workflow between a requester and a session host. A compound index on `{ session: 1, requester: 1, status: 1 }` prevents a student from spamming duplicate pending requests on the same session.

Field Name	BSON Type	Constraints	Indexing	Functional Description
_id	ObjectId	Primary Key (Auto)	Unique Index	Unique booking request identifier.
session	ObjectId	Required, Ref: 'Session'	Compound Index	Foreign reference to the target session offering.
requester	ObjectId	Required, Ref: 'User'	Compound Index	Foreign reference to the student requesting the slot.
creator	ObjectId	Required, Ref: 'User'	Indexed	Foreign reference to the session host (denormalized).
message	String	Optional, Max: 500	None	Introductory note stating learning goals and trade offer.
status	String	Enum: [Pending, Accepted, Rejected, Cancelled]	Indexed	Request state machine indicator (Default: 'Pending').
createdAt / updatedAt	Date	Auto Timestamps	Indexed	Submission and decision audit timestamps.

5. RESTful API Endpoint Documentation

The TimeTrade REST API conforms to strict RESTful conventions with predictable resource URIs, standard HTTP verbs, structured JSON error formats, and semantic status codes.

Method	Endpoint Route	Auth	Request Body / Params	Status	Response Summary
POST	/api/auth/register	Public	{ name, email, password, department, bio }	201	Returns user profile object and signed JWT token.
POST	/api/auth/login	Public	{ email, password }	200	Validates credentials; returns user profile and JWT token.
GET	/api/auth/me	Bearer JWT	None (Header Token)	200	Returns current authenticated student user profile.
PUT	/api/auth/profile	Bearer JWT	{ name, department, bio, avatarUrl }	200	Updates profile metadata and returns updated user.
GET	/api/sessions	Public	Query: ?search=&category=&status=&page=&limit=	200	Paginated list of sessions with populated creator data.
GET	/api/sessions/my	Bearer JWT	None	200	List of all session offerings published by the logged-in user.
GET	/api/sessions/:id	Public	Param: id (MongoDB ObjectId)	200	Detailed session object with creator profile and bio.
POST	/api/sessions	Bearer JWT	{ title, category, description, durationMinutes, timeSlot }	201	Creates offering, sets status to 'Open', returns session.
PUT	/api/sessions/:id	Bearer JWT	{ title, category, description, status, ... }	200	Updates offering (verified: creator only).
DELETE	/api/sessions/:id	Bearer JWT	Param: id	200	Deletes offering and cascade purges associated bookings.
POST	/api/bookings	Bearer JWT	{ sessionId, message }	201	Submits slot request; validates session is open

					and not self-owned.
GET	/api/bookings/incoming	Bearer JWT	None	200	Returns incoming requests for user's offerings with requester details.
GET	/api/bookings/my	Bearer JWT	None	200	Returns outgoing requests submitted by current student.
PATCH	/api/bookings/:id/status	Bearer JWT	{ status: 'Accepted' | 'Rejected' }	200	Host accepts/rejects. On accept: session -> 'Booked'.
PATCH	/api/bookings/:id/cancel	Bearer JWT	Param: id	200	Cancels request. If previously accepted, reverts session to 'Open'.
GET	/api/health	Public	None	200	Returns API uptime, timestamp, and MongoDB connection status.

6. Implementation & Key Code Modules

6.1 JWT Authentication & Route Protection Middleware

Authentication is secured through asymmetric-ready HMAC-SHA256 JWT tokens. The backend protect middleware extracts the token from the HTTP Authorization header, verifies signature integrity, and attaches the active User document to req.user. On the client side, ProtectedRoute intercepts unauthenticated navigation attempts and redirects to /login while preserving the intended destination in router location state.

```
// Code Module: JavaScript (Backend JWT Middleware)

// File: /server/middleware/authMiddleware.js
export const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return errorResponse(res, 'Authentication required. No token provided.', 401);
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');
    if (!user) return errorResponse(res, 'User no longer exists.', 401);
    req.user = user;
    next();
  } catch (error) {
```

```
    if (error.name === 'TokenExpiredError') {
      return errorResponse(res, 'Session token expired. Please sign in again.', 401);
    }
    return errorResponse(res, 'Invalid authentication token.', 401);
  }
};
```

6.2 Two-Sided Request Approval State Machine

The core domain innovation resides in bookingService.js. When a host accepts an incoming slot request, the service performs an atomic transition: the chosen booking is marked 'Accepted', the parent Session status is switched to 'Booked', and all other pending inquiries for that session are automatically updated to 'Rejected'. Conversely, if an accepted booking is cancelled, the session availability is seamlessly restored to 'Open'.

```
// Code Module: JavaScript (Two-Sided Approval State Machine)
// File: /server/services/bookingService.js (State Machine Logic)
export const updateBookingStatus = async ({ bookingId, userId, newStatus }) => {
  const booking = await Booking.findById(bookingId);
  if (!booking) throw new Error('Booking request not found.');
  if (booking.creator.toString() !== userId.toString()) {
    throw new Error('Forbidden: Only the session host can update request status.');
  }

  booking.status = newStatus;
  await booking.save();

  if (newStatus === 'Accepted') {
    // 1. Mark corresponding session as 'Booked'
    await Session.findByIdAndUpdate(booking.session, { status: 'Booked' });

    // 2. Automatically reject all other competing pending requests
    await Booking.updateMany(
      { session: booking.session, _id: { $ne: booking._id }, status: 'Pending' },
      { status: 'Rejected' }
    );
  }
  return await booking.populate([
    { path: 'session', select: 'title category durationMinutes timeSlot status' },
    { path: 'requester', select: 'name email department avatarUrl' }
  ]);
};
```

6.3 Axios Client Interceptor Pipeline

To guarantee seamless client-server interaction without manual token handling in components, a centralized Axios instance intercepts every outbound request and inbound response. It injects authentication tokens and dispatches a custom window event on 401 errors to trigger immediate, graceful logout.

```
// Code Module: JavaScript (Axios Interceptors)
// File: /client/src/api/axios.js (Network Layer Interceptors)
const api = axios.create({ baseURL: import.meta.env.VITE_API_BASE_URL || '/api' });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('timetrade_token');
```

```
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('timetrade_token');
      localStorage.removeItem('timetrade_user');
      window.dispatchEvent(new Event('timetrade_auth_logout'));
    }
    return Promise.reject(error);
  }
);
export default api;
```

7. Security & Production Readiness

TimeTrade enforces comprehensive defense-in-depth security measures to protect student data and prevent common web application vulnerabilities as defined by OWASP Top 10.

Security Vector	Implemented Mechanism	Production Justification
Password Encryption	bcryptjs with 10 salt rounds	Protects user credentials against rainbow table and brute-force cracking using computationally intensive key derivation.
Brute-Force Mitigation	express-rate-limit (30 req / 15 min)	Enforces IP-based rate limiting on /api/auth endpoints, preventing automated credential stuffing attacks.
HTTP Header Hardening	Helmet.js middleware	Sets Content-Security-Policy, X-Frame-Options (anti-clickjacking), X-Content-Type-Options, and Strict-Transport-Security.
Origin Whitelisting	Express CORS validation	Restricts cross-origin AJAX access exclusively to authorized frontend domains specified in environment configuration.
Input Sanitization	express-validator body schemas	Enforces strict type, length, and format constraints on all incoming JSON payloads, neutralizing NoSQL injection vectors.
Operational Error Isolation	Centralized errorHandler middleware	Traps Mongoose CastErrors, duplicate key collisions, and runtime exceptions without exposing internal server stack traces to clients.
Git Repository Hygiene	Triple-tier .gitignore configuration	Ensures .env secrets, node_modules/, and build artifacts are strictly excluded from version control.

8. Testing & Quality Verification

8.1 API Integration Test Matrix

System endpoints were validated using Postman and automated test suites covering positive, boundary, and negative error scenarios.

Test ID	Target Endpoint	Test Scenario & Input	Expected Status	Observed Result
TC-AUTH-01	POST /api/auth/register	Valid student registration data	201 Created	PASS - User saved, JWT returned.
TC-AUTH-02	POST /api/auth/register	Duplicate email registration	409 Conflict	PASS - Handled with duplicate alert message.
TC-AUTH-03	POST /api/auth/login	Invalid password attempt	401 Unauthorized	PASS - Access denied, generic error returned.
TC-SESS-01	GET /api/sessions	Query with category='Coding' & status='Open'	200 OK	PASS - Returns filtered session list.
TC-SESS-02	POST /api/sessions	Unauthenticated session creation attempt	401 Unauthorized	PASS - Blocked by JWT protect middleware.
TC-BOOK-01	POST /api/bookings	Student requests own offering	400 Bad Request	PASS - Prevented by self-booking guard.
TC-BOOK-02	PATCH /api/bookings/:id/status	Host accepts request; conflicting requests exist	200 OK	PASS - Session becomes 'Booked', competitor rejected.
TC-BOOK-03	PATCH /api/bookings/:id/cancel	Accepted booking cancelled by requester	200 OK	PASS - Session reverts to 'Open'.

8.2 User Acceptance Testing (UAT)

- **Scenario A (Student Discovery Walkthrough):** A student visits the marketplace, applies the 'Notes' topic filter, locates 'BCA Semester 5 Web Technologies Handwritten Notes', views host details, and submits a request note in under 45 seconds.
- **Scenario B (Host Fulfillment Walkthrough):** The session host logs into their dashboard, sees an incoming request with an amber badge indicator, reviews the requester's note, clicks 'Accept', and verifies the session is immediately marked 'Booked'.
- **Scenario C (Session Lifecycle Completion):** After completing the collaborative notes review, the host clicks 'Mark Completed' in the dashboard, transferring the record to the immutable 'Completed History' log.

9. Conclusion & Future Enhancements

9.1 Project Outcomes & Academic Conclusion

The TimeTrade: Student Time-Exchange Board project demonstrates a comprehensive, enterprise-ready implementation of modern full-stack web engineering principles for the BCA Semester 5 Capstone curriculum. By successfully unifying a decoupled React 18 Single Page Application with a secured Node.js/Express REST backend and MongoDB document storage, the platform bridges the critical gap between academic peer support needs and structured collaboration. All functional requirements—including

user onboarding, full CRUD offerings, debounced marketplace search, and the two-sided slot reservation state machine—have been realized and verified with zero production errors.

9.2 Future Enhancement Roadmap

- **Real-Time WebSocket Chat (Socket.io):** Embed integrated peer messaging allowing students to chat and clarify session logistics directly within the slot request modal prior to host acceptance.
- **Integrated WebRTC Video Study Rooms:** Provide one-click peer-to-peer audio/video rooms with screen sharing directly inside the TimeTrade browser interface for remote pair programming.
- **Calendar Synchronization (.ics / Google Calendar API):** Enable automatic export of accepted session schedules into students' personal calendars with automated reminder alerts.
- **Time Credit Wallet & Student Karma System:** Introduce a digital credit ledger tracking time balances, accompanied by student review ratings (1-5 stars) and skill endorsements.
- **Automated Push & Email Notifications:** Implement transactional email notifications (via Nodemailer/SendGrid) alerting hosts when new slot requests arrive.

10. References & Appendix

10.1 Technical & Academic References

- **React Documentation:** Meta Platforms Inc., 'React 18 & Hooks Specification', https://react.dev
- **Express.js Foundation:** StrongLoop / OpenJS Foundation, 'Express 4 REST Framework', https://expressjs.com
- **MongoDB Mongoose ODM:** Automattic Inc., 'Mongoose v8 Schema Modeling Reference', https://mongoosejs.com
- **Tailwind CSS:** Tailwind Labs, 'Utility-First CSS Framework Documentation', https://tailwindcss.com
- **OWASP Foundation:** Open Web Application Security Project, 'OWASP Top 10 Web Application Security Risks', https://owasp.org
- **IETF RFC 7519:** Jones et al., 'JSON Web Token (JWT) Standard Specification', Internet Engineering Task Force

10.2 Project Repository & Local Setup Summary

The complete source code is organized within a dual-module repository. Follow these commands to initialize and launch the platform locally:

```
// Code Module: Bash / Terminal Setup

# 1. Clone repository & install dependencies
git clone <repository-url>
cd timetrade-platform
npm install
npm run install:all

# 2. Seed database with realistic demo students & offerings
npm run seed

# 3. Launch both backend (port 5000) & frontend (port 5173) concurrently
npm run dev
```

```
# 4. Access application in browser
# Frontend: http://localhost:5173
# REST API: http://localhost:5000/api/health
```